

ورشة GIT وGITHUB

مزامنة الفريق: كيف أرى تعديلات زملائي؟

Session 04 · Sync, Merge & Undo

- «زميلي أضاف تعديلاً... وأنا لا أراه!» — لماذا؟
- ثلاثة أسباب شائعة: لم يرفع (push)، أو رفع على فرع لم يدمج، أو نسختك لم تُحدَّث (pull)
- اليوم نتقن الطرف الثالث ونفهم كيف يعمل الدمج فعلياً
- ونضيف مهارة أهم: **التراجع الآمن** عندما يفسد شيء

هدف اليوم ليس تنفيذ أوامر، بل **قراءة** مخرجاتها: أين أنا؟ ما الجديد؟ ما الفرق؟ من فعل هذا؟

الفرق في خطوة واحدة — pull مقابل fetch



لترى ما سيحدث قبل أن يحدث `git log HEAD..origin/main` ثم `git fetch`: العادة الاحترافية

```
$ git switch main
$ git fetch origin
remote: Enumerating objects: 9, done.
From https://github.com/<org>/class-team-hub
   a1b2c3d..f9e8d7c  main       → origin/main

$ git status -sb
## main...origin/main [behind 2]

$ git log --oneline HEAD..origin/main
f9e8d7c feat(members): add card for leen-docs
3c7d9e1 Merge pull request #7 from nour-dev/feature/nour-card
```

- `behind 2` = عند الزملاء التزامان لم تجلبهما بعد
- العادة الاحترافية: افحص بـ `fetch` ثم اقرأ، ثم ادمج بوعي

الحالة في git status	المعنى	الأمر السليم
ahead 2	عندك التزامات غير مرفوعة	git push
behind 3	عند الزملاء التزامات لم تجلبها	git pull --ff-only
ahead 1, behind 3	التاريخان اختلفا	git pull --rebase
diverged	اقتراق ثم محاولة دفع	push ثم pull --rebase

تصفية الحالة: `git status -sb` — سطر واحد يخبرك بكل شيء، قبل أي أمر آخر.

1) دمج سريع (لا عمل متبادل):

```
$ git merge --ff-only origin/main
```

```
Updating a1b2c3d..f9e8d7c
```

```
Fast-forward
```

2) دمج بعقدة (عمل متبادل):

```
$ git merge feature/omar-qa-card
```

```
Merge made by the 'ort' strategy.
```

```
1 file changed, 9 insertions(+)
```

3) الدمج المضغوط:

```
$ git merge --squash feature/sara-fixes
```

```
Squash commit -- not updating HEAD
```

```
$ git commit -m "feat(ui): improve member card layout (#9)"
```

الاستراتيجية	شكل التاريخ	متى نستخدمها
Merge commit	فروع وُعقد واضحة	فرع طويل بمهام متعددة
Squash and merge	الالتزام واحد نظيف	فرع صغير أو إصلاحات متتابة
Rebase and merge	خط مستقيم بلا عُقد	تاريخ خطّي للمراجعة

القاعدة: **rebase للفروع الشخصية فقط، merge للفروع المشتركة.** إعادة تشكيل فرع يعمل عليه زميل تُتلف عمله.

```
$ git log --oneline --all --graph --decorate -8 # شجرة الفريق
$ git log -p -- data/members/leen-docs.json    # تاريخ ملف واحد
$ git blame data/members/leen-docs.json        # من كتب كل سطر
$ git shortlog -sn --all                       # ترتيب المساهمين
$ git diff main..feature/x --stat              # ما سيدخل فعلاً
```

- سؤال: أي ملف عدّله **كل** الأعضاء؟ الجواب: `data/members-index.json`
- ولهذا هو مُولّد آلياً: لا يُكتب يدوياً ولا يُحلّ تعارضه يدوياً

الحالة	الأمر الصحيح	يُحذف عملاً؟
تعديل غير مُلتزم	<code><git restore <file</code>	نعم (هو المقصود)
ملف مُمرّر للمرحلة خطأً	<code><git restore --staged <file</code>	لا
آخر التزام لم يُرفع	<code>git commit --amend</code>	لا
الالتزام مرفوع خاطئ	<code><git revert <sha</code>	لا (يعكسه)
تحويل فرع وعملك ناقص	<code>git stash</code> ثم <code>git stash pop</code>	لا
حذفت شيئاً ولا تدري	<code>git reflog</code> ثم <code>switch -c rescue</code>	لا

```
$ git revert --no-edit 8de1e7a
[main 6f7e8d9] Revert "feat(ui): change card colors"

$ git log --oneline -3
6f7e8d9 (HEAD → main) Revert "feat(ui): change card colors"
2b3c4d5 docs(readme): add FAQ section
8de1e7a feat(ui): change card colors
```

- `revert` ينشئ التزاماً جديداً يعكس الأول ← التاريخ يبقى صحيحاً لكل من سحب المشروع
- `reset --hard` يحذف الالتزامات ← من بنى عليها عملاً ينهار مشروعه

```
$ git reflog -5
6f7e8d9 HEAD@{0}: revert: Revert "feat(ui): change card colors"
2b3c4d5 HEAD@{1}: commit (amend): docs(readme): add FAQ section
1e1682a HEAD@{2}: commit: docs(readme): add faq
5aaeef2 HEAD@{3}: reset: moving to origin/main

$ git switch -c rescue 5aaeef2      # استرجاع أي حالة سابقة
```

reflog سجل محلي لا يُرفع إلى GitHub — فهو ينفذك على جهازك، لكنه لا ينفذ زميلاً على جهاز آخر.

```
$ git branch --merged main      # الفروع التي اندمج عملها فعلاً
feature/ali-card
feature/leen-card
* main

$ git branch -d feature/ali-card feature/leen-card
Deleted branch feature/ali-card (was 1a2b3c4).

$ git fetch --prune
- [deleted]          (none)      → origin/feature/ali-card
```

احذف الفروع المدموجة فقط. الفرع غير المدموج يحتاج `-D` (حذف قسري) — لا تستخدمه إلا بعد التأكد أن عملك محفوظ أو مرفوع.

- `main` عندك محدّث ومساوٍ لـ `origin/main`
- نفّذت الدمج بالاستراتيجيات الثلاث وتعرف الفرق بينها عملياً
- جرّبت `stash` و `amend` و `revert` و `reflog`
- تقرير الفريق مملوء بالأرقام الحقيقية من المستودع
- الجلسة القادمة: **التعارضات والنشر** — سنصنع تعارضاً عن قصد ونحلّه، ثم ننشر الموقع للعالم

تذكّر: التعارض ليس خطأً. هو رسالة من Git تقول «هنا تعديّلان، اختر أنت» — والجلسة القادمة تجعل هذا الاختيار سهلاً.